UNIVERSIDADE / DEPARTAMENTO DE COMPUTAÇÃO
DISCIPLINA: BANCO DE DADOS AVANÇADO / SISTEMAS DISTRIBUÍDOS

Seminário Acadêmico & Laboratório Prático:
Apache CouchDB vs. Couchbase Server

Dossiê Técnico Completo: Fundamentos Arquiteturais, Roteiros de Fala dos 7 Integrantes, Laboratório com Docker, Matriz Comparativa e Avaliação Didática.

Duração Prevista:	35 a 45 minutos no total
Equipe de Alunos:	7 Integrantes (fala equilibrada de ~5 a 6 min/integrante)
Público-Alvo:	Graduação em Ciência da Computação / Engenharia de Software
Repositório Online:	https://github.com/PHLopesvc07/ApresentacaoBD
Apresentação Web:	https://phlopesvc07.github.io/ApresentacaoBD/
Slides PowerPoint:	Apresentacao_CouchDB_vs_Couchbase.pptx (16:9 Dark Mode)

Estrutura do Dossiê para Avaliação Docente:

- **Módulo 1:** Roteiro de Fala e Estrutura dos 18 Slides Técnicos (Integrantes 1 a 7)
- **Módulo 2:** Matriz Comparativa Completa (CAP, PACELC, MVCC, N1QL e Operação)
- **Módulo 3:** Guia do Workshop Prático (Docker Compose, cURL, N1QL e Script Python)
- **Módulo 4:** Avaliação Didática (5 Questões Universitárias Inéditas com Gabarito Comentado)
- **Módulo 5:** Plano Operacional de Contingência e Ensaio Cronometrado (48 Horas)

MÓDULO 1: PESQUISA, SLIDES E ROTEIROS DE FALA

Planejamento estrito de fala para os 7 integrantes da equipe, mantendo tempo equilibrado e postura técnica universitária.

Integrante	Slides	Tema Responsável	Tempo Est.
Integrante 1	1, 2 e 3	Origem Histórica, Genealogia (Membase + CouchOne) e Bifurcação Filosófica	~6 min
Integrante 2	4, 5 e 6	Arquitetura Apache CouchDB: Append-Only B+Tree, MVCC (_rev), Views e _changes	~6 min
Integrante 3	7, 8 e 9	Arquitetura Couchbase: Memory-First, vBuckets, MDS (Multi-Service) e N1QL/GSI	~6 min
Integrante 4	10 e 11	Casos de Uso e Mercado CouchDB: PouchDB Offline-First, IoT e Trade-offs	~5m30s
Integrante 5	12 e 13	Casos de Uso e Mercado Couchbase: Escala Extrema, XDCR, LinkedIn e Trade-offs	~5m30s
Integrante 6	14 e 15	Matriz Comparativa Técnica Direta: Teorema CAP/PACELC, Latência e Operação	~5m30s
Integrante 7	16, 17 e 18	Workshop Prático ao Vivo: Docker Compose, REST API, SQL++ e Script Python	~6m30s

Integrante 1 — Slides 1 a 3 (Fundamentos e Genealogia Histórica)

Slide 1: Capa e Contextualização do Problema

- A crise do RDBMS clássico na Web 2.0 e limitações de escala vertical (Scale-Up).
- Impedância objeto-relacional e o custo de bloqueios pessimistas distribuídos.
- O modelo documental em JSON e a divergência entre dois bancos com nomes similares.

Roteiro Falado (Script): *"Bom dia/Boa tarde a todos. Hoje nossa equipe vai apresentar uma análise arquitetural profunda sobre dois bancos de dados NoSQL orientados a documentos que frequentemente causam confusão pelo nome, mas que resolveram problemas completamente distintos: o Apache CouchDB e o Couchbase. Quando olhamos para as limitações dos bancos relacionais tradicionais em sistemas distribuídos de larga escala, esbarramos em dois gargalos clássicos: a impedância objeto-relacional e o custo de manter transações com bloqueio pessimista. CouchDB e Couchbase adotam JSON, mas enquanto um nasceu para dominar redes instáveis e offline-first, o outro foi desenhado para latência sub-milissegundo em memória."*

Slide 2: A Fusão de 2011 e a Grande Divergência

- 2005: Damien Katz cria o CouchDB (Cluster Of Unreliable Commodity Hardware) em Erlang.
- 2010: Criação do Membase (Memcached + persistência + vBuckets).
- Fevereiro 2011: Fusão CouchOne + Membase = Couchbase Inc.
- Divergência: Couchbase adotou motor em RAM do Membase; Apache CouchDB manteve motor Erlang na ASF.

Roteiro Falado (Script): *"Para entender por que ambos têm 'Couch' no nome, voltamos a 2005. O CouchDB foi criado por Damien Katz com foco em hardware comum. Em paralelo, criadores do Memcached criaram o Membase. Em 2011, fundiram-se na Couchbase Inc. O ponto vital é: o produto Couchbase Server não é o CouchDB renomeado. A engenharia fundiu o motor em memória do Membase com JSON, enquanto a Apache Software Foundation manteve o CouchDB independente."*

Slide 3: Bifurcação Filosófica Fundamental

- Apache CouchDB: Protocolo nativo HTTP/REST (porta 5984), caminho Disk-First (append-only) e MVCC explícito via _rev.
- Couchbase: Protocolo binário TCP (porta 11210), caminho Memory-First (RAM), consistência por vBucket (CP) e N1QL (SQL++).

Roteiro Falado (Script): *"Essa separação gerou duas filosofias opostas. O Apache CouchDB fala HTTP nativamente, grava em disco append-only e expõe o campo _rev para concorrência otimista e sincronização offline. O Couchbase opera em protocolo binário TCP, colocando um cache gerenciado em RAM na frente do disco para confirmar operações em frações de milissegundo. Passo agora ao Integrante 2 para dissecar o CouchDB."*

Integrante 2 — Slides 4 a 6 (Arquitetura Interna do Apache CouchDB)

Slide 4: Motor Append-Only e B+Tree Storage

- Escrito em Erlang/OTP: Concorrência massiva e isolamento de falhas (Let it crash).
- Zero in-place updates: Modificações vão para o fim do arquivo .couch junto com o Tail Header.
- Imunidade a corrupção em falhas elétricas e necessidade periódica de Compaction.

Roteiro Falado (Script): *"O motor de armazenamento do CouchDB em Erlang é uma B+Tree Append-Only. Ele nunca sobrescreve blocos existentes; novas revisões e nós modificados vão para o final do arquivo com o novo Tail Header. Isso traz duas vantagens fundamentais: leituras nunca bloqueiam escritas, e o banco é virtualmente imune a corrupções por queda de energia. O contraponto é a necessidade de Compactação para limpar dados órfãos."*

Slide 5: Controle MVCC (_rev) e Árvore de Revisões

- Identificador de revisão no formato -.
- Atualizações exigem _rev atual da ponta da folha; divergências disparam HTTP 409 Conflict.
- Conflitos em replicação offline bifurcam a árvore; algoritmo determinístico elege o vencedor sem NTP.

Roteiro Falado (Script): *"O CouchDB garante consistência através de MVCC no documento. Para atualizar um registro, a aplicação deve enviar o _rev atual; caso contrário, recebe HTTP 409 Conflict. Em replicação desconectada, se dois nós editarem o mesmo registro, o CouchDB preserva ambas as revisões e usa um algoritmo determinístico baseado no hash para escolher a mesma versão vencedora em todos os nós."*

Slide 6: Views MapReduce, Mango Query e _changes

- Views em Design Documents (_design/): Funções map(doc) em JS emitindo chaves para B-Trees.
- Mango Queries (/_find): Sintaxe declarativa em JSON introduzida no CouchDB 2.0+.
- Endpoint /_changes: Log sequencial de mutações que alimenta o protocolo de replicação contínua.

Roteiro Falado (Script): *"Para consultas, temos Views MapReduce e Mango Queries declarativas em JSON. O grande trunfo é o endpoint _changes: um log sequencial de alterações em tempo real. O protocolo de replicação lê esse feed, compara revisões via _revs_diff e transfere apenas as diferenças em lote. Passo agora a palavra ao Integrante 3."*

Integrante 3 — Slides 7 a 9 (Arquitetura Interna do Couchbase Server)

Slide 7: Arquitetura Memory-First e 1024 vBuckets

- Data Service (KV): Leituras e escritas servidas diretamente do Managed Cache em RAM.
- Protocolo interno DCP (Database Change Protocol): Stream binário em memória para replicação e disco.
- Particionamento fixo em 1024 vBuckets: SDK calcula CRC32(Key) % 1024 e conecta via Single-Hop.

Roteiro Falado (Script): *"No Couchbase, o caminho de dados é Memory-First. As escritas vão para a RAM gerenciada pelo Data Service e retornam confirmação imediata em microssegundos, enquanto o protocolo DCP propaga os dados para réplicas e disco. O bucket é fatiado em 1024 vBuckets estáticos; o SDK calcula o hash CRC32 e conecta diretamente no nó ativo daquele vBucket sem proxies intermediários."*

Slide 8: Multi-Dimensional Scaling (MDS) e Isolamento

- Fim do vizinho barulhento (Noisy Neighbor) típico de nós homogêneos.
- Desacoplamento em 6 serviços: Data (RAM), Index (NVMe), Query (CPU), Search, Analytics e Eventing.
- Escala independente de hardware conforme o gargalo real de cada workload.

Roteiro Falado (Script): *"Em bancos NoSQL clássicos, todos os nós fazem tudo. No Couchbase, o Multi-Dimensional Scaling separa o cluster em serviços independentes: Data com muita RAM, Query com muita CPU e Index com SSDs ultrarrápidos. Mutações fluem via memória pelo protocolo DCP sem que consultas analíticas pesadas roubem CPU das leituras de chave-valor."*

Slide 9: Scopes, Collections, N1QL (SQL++) e GSI

- Mapeamento relacional: Bucket (DB) -> Scope (Schema) -> Collection (Tabela).
- N1QL / SQL++: ANSI SQL completo para JSON com suporte a JOIN, UNNEST, CTEs e transações ACID.
- Global Secondary Indexes (GSI): Evita consultas lentas do tipo Scatter-Gather.

Roteiro Falado (Script): *"O Couchbase organiza dados em Bucket, Scope e Collection. Sua linguagem N1QL (SQL++) permite consultas ANSI SQL em JSON, incluindo JOINs e o operador UNNEST para desmembrar arrays. Para garantir que o SQL++ seja veloz, os índices GSI ficam centralizados nos nós de indexação, respondendo queries em milissegundos. Passo ao Integrante 4."*

Integrante 4 — Slides 10 e 11 (Casos de Mercado e Trade-offs do CouchDB)

Slide 10: O Rei do Offline-First (CouchDB + PouchDB)

- PouchDB no cliente: Executa no navegador ou app móvel com sincronização bidirecional em 1 linha.
- Zero latência percebida: Aplicações operam sem rede e sincronizam deltas quando houver conexão.
- Cenários ideais: Mineração, saúde em zonas rurais, aviação, plataformas offshore e IoT.

Roteiro Falado (Script): *"Quando escolher o CouchDB? Quando a premissa é Offline-First e borda. Pareado ao PouchDB, ele permite que aplicações web e mobile funcionem perfeitamente sem sinal de internet. Imagine equipes de saúde em aldeias remotas ou técnicos em minas: eles coletam dados offline e, ao obter sinal por 30 segundos, o banco sincroniza com a nuvem automaticamente."*

Slide 11: Cases Reais e Limitações do CouchDB

- Casos reais: Hyperledger Fabric (World State oficial em JSON), npm registry e eHealth Africa.
- Trade-offs: Inadequado para contadores de alta frequência na mesma chave e escritas massivas contínuas.
- Falta de JOINs no servidor e necessidade de monitoramento de espaço em disco e compactação.

Roteiro Falado (Script): *"O CouchDB é usado pelo Hyperledger Fabric da Linux Foundation como banco oficial de estado de contratos inteligentes em JSON auditável. Porém, não use o CouchDB para telemetria de alta frequência que atualiza o mesmo documento 500 vezes por segundo, pois a árvore de revisões e o arquivo append-only inflarão com rapidez. Passo ao Integrante 5."*

Integrante 5 — Slides 12 e 13 (Casos de Mercado e Trade-offs do Couchbase)

Slide 12: Escala Extrema e Replicação Global XDCR

- Eliminação da arquitetura dupla: Substitui a combinação complexa RDBMS/MongoDB + Redis.
- XDCR (Cross Data Center Replication): Stream binário memória-a-memória entre datacenters/nuvens.
- Suporte a topologias Active-Active e resolução de conflitos via HLC (Hybrid Logical Clocks).

Roteiro Falado (Script): *"O Couchbase brilha em missões críticas de altíssima escala. Ele elimina a complexidade de manter um banco no fundo e um cluster de Redis na frente. Para escala global, o XDCR replica dados diretamente da memória RAM de um datacenter no Brasil para outro nos Estados Unidos via protocolo DCP, resolvendo conflitos em tempo real com relógios lógicos híbridos."*

Slide 13: Cases Globais e Limitações do Couchbase

- Cases reais: LinkedIn (10 milhões de queries/s com < 2ms), Amadeus/Sabre (aviação global) e PayPal.
- Trade-offs: Alto custo de hardware (exige dezenas de GBs de RAM) e complexidade em Day-2 Ops.
- Inviável para microsserviços pequenos em VMs de 512 MB/1 GB.

Roteiro Falado (Script): *"Os números do Couchbase impressionam: o LinkedIn processa 10 milhões de queries por segundo, e empresas como Amadeus gerenciam reservas de voos no mundo todo. Mas o custo é alto: ele exige servidores robustos com muita RAM e planejamento refinado de serviços MDS. Passo a palavra ao Integrante 6 para a síntese comparativa."*

Integrante 6 — Slides 14 e 15 (Matriz Comparativa Técnica Direta)

Slide 14: Comparativo de Modelo, CAP e Consultas

- Modelo: CouchDB (Databases planos) vs. Couchbase (Bucket ➔ Scope ➔ Collection).
- Teorema CAP: CouchDB é AP (disponibilidade contínua); Couchbase é CP por padrão (consistência por vBucket).
- Consultas: Views/Mango sem JOIN no CouchDB vs. SQL++ com ANSI JOIN e GSI no Couchbase.

Roteiro Falado (Script): *"Sintetizando os dois bancos em nossa matriz comparativa: no modelo de dados, o Couchbase espelha o relacional em Bucket, Scope e Collection. Sob o Teorema CAP, o CouchDB prioriza disponibilidade (AP), enquanto o Couchbase foca em consistência imediata servida pelo nó ativo do vBucket (CP por padrão). Em consultas, o Couchbase entrega o poder do SQL++."*

Slide 15: Comparativo de Replicação, Latência e Operação

- Replicação: HTTP/REST no CouchDB (PouchDB amigável) vs. TCP binário DCP/XDCR em RAM no Couchbase.
- Latência: Casa de milissegundos no CouchDB vs. Sub-milissegundo em microssegundos no Couchbase.
- Conclusão de decisão técnica: Conectividade instável pede CouchDB; throughput e latência crítica pedem Couchbase.

Roteiro Falado (Script): *"Nos aspectos operacionais: a replicação do CouchDB sobre HTTP permite integração simples com navegadores. O Couchbase opera em protocolo binário TCP direto da memória, atingindo latência sub-milissegundo. Em suma: se o desafio é rede e offline-first, vá de CouchDB; se o desafio é vazão e latência, vá de Couchbase. Passo ao Integrante 7 para o workshop prático."*

Integrante 7 — Slides 16 a 18 (Workshop Prático e Demonstração ao Vivo)

Slide 16: Topologia do Laboratório Docker Compose

- Container 1: apache/couchdb:3.3 exposto na porta 5984 (API REST e interface Fauxton /_utils).
- Container 2: couchbase:community-7.6.0 com portas 8091 (Web Console), 8093 (Query) e 11210 (KV).

Roteiro Falado (Script): *"Agora vamos ver a arquitetura em execução no laboratório Docker Compose. O CouchDB requer apenas a porta 5984 para API e interface web Fauxton. Já o Couchbase expõe a porta 8091 para administração, 8093 para queries N1QL e 11210 para o acesso binário dos SDKs aos vBuckets."*

Slide 17: Live Demo 1 — Apache CouchDB (MVCC e _changes)

- Demonstração de inserção via cURL e falha com HTTP 409 Conflict ao omitir o _rev.
- Atualização com sucesso ao enviar a revisão correta e inspeção do stream contínuo /_changes.

Roteiro Falado (Script): *"Na primeira demonstração, criamos o aluno_01 no CouchDB. Ao tentar atualizar a nota sem o _rev atual, o servidor devolve na hora o erro HTTP 409 Document update conflict, provando o MVCC ao vivo. Passando o _rev correto, o dado atualiza e o evento salta no terminal que escuta o feed contínuo do endpoint _changes."*

Slide 18: Live Demo 2 — Couchbase SQL++ e Automação Python

- Criação de índice GSI, execução de query SQL++ com UNNEST em 1.4 ms e script Python CRUD completo.
- Verificação de 0% Cache Miss Ratio (leitura 100% da RAM) e abertura para dinâmica com a turma.

Roteiro Falado (Script): *"No Couchbase, entramos no Query Workbench, criamos o índice secundário GSI e executamos uma consulta com UNNEST sobre o array de disciplinas. A resposta retorna em 1.4 milissegundos direto da RAM. Concluímos com nosso script Python demonstrando o tratamento de concorrência em código real e abrindo para a avaliação!"*

MÓDULO 2: MATRIZ COMPARATIVA TÉCNICA DIRETA

Confronto detalhado das garantias arquiteturais sob as lentes do Teorema CAP, teorema PACELC e engenharia de dados.

Dimensão Técnica	Apache CouchDB	Couchbase Server
Modelo de Dados	Documentos JSON + Anexos binários (_attachments). Estrutura plana de Databases sem schemas intermediários.	JSON + Chave-Valor Binário. Hierarquia em 3 níveis: Bucket (DB) ➔ Scope (Schema) ➔ Collection (Tabela).
Teorema CAP & PACELC	**AP (PC/EC):** Prioriza disponibilidade contínua e tolerância a partições. Consistência eventual com resolução determinística de conflitos.	**CP por padrão (PC/EC):** Um único nó ativo detém a posse do vBucket por chave, garantindo consistência imediata (Read-Your-Own-Writes) em RAM.
Concorrência & ACID	MVCC otimista explícito via árvore de revisões (_rev). Sem suporte a transações multi-documento no servidor.	CAS (Compare-And-Swap) otimista ou bloqueio pessimista + Transações ACID Distribuídas Multi-Documento (v6.5+).
Mecanismos de Consulta	Views pré-computadas em B-Tree (MapReduce em JS/Erlang) + Mango Query (POST /_find) declarativo em JSON. Não possui JOINs.	**N1QL / SQL++:** ANSI SQL completo para JSON com suporte nativo a INNER/LEFT JOIN, UNNEST (arrays), GROUP BY e índices GSI.
Mecanismos de Replicação	Couch Replication Protocol sobre HTTP/REST (_changes + _revs_diff). Nativo para peer-to-peer, offline e navegadores (PouchDB).	Protocolo DCP (intra-cluster) e XDCR (inter-cluster) via stream binário TCP de memória para memória. Mobile requer Sync Gateway.
Performance e Latência	Casa dos **milissegundos (5ms a 30ms+)**, limitada pelo I/O append-only de disco e serialização de cabeçalhos HTTP.	Latência **sub-milissegundo (< 1ms / microssegundos)** servida diretamente do cache gerenciado em RAM para operações de chave.
Complexidade Operacional	Baixa pegada de memória. Roda com < 128 MB de RAM. Exige rotina de Compaction para liberar espaço em disco de revisões órfãs.	Exige servidores robustos (16 GB a 64 GB+ de RAM por nó), planejamento de serviços MDS e rebalanceamento de vBuckets.

MÓDULO 3: WORKSHOP PRÁTICO (PASSO A PASSO REPRODUZÍVEL)

Guia de comandos testados e funcionais utilizando Docker Compose para execução em laboratório universitário.

1. Orquestração com Docker Compose (docker-compose.yml)

Sobe simultaneamente o Apache CouchDB 3.3 e o Couchbase Server 7.6 Community Edition:

```
version: '3.8'
services:
  couchdb:
    image: apache/couchdb:3.3
    container_name: lab_couchdb
    ports:
      - "5984:5984" # API HTTP REST + Interface Fauxton (/_utils)
    environment:
      - COUCHDB_USER=admin
      - COUCHDB_PASSWORD=admin123

  couchbase:
    image: couchbase:community-7.6.0
    container_name: lab_couchbase
    ports:
      - "8091-8096:8091-8096" # Web Console (8091), N1QL Query (8093)
      - "11210:11210"         # Data Service KV Binário (vBuckets)
```

2. Comandos de Inicialização dos Clusters

```
# Subir os containers em background
docker compose up -d

# Inicializar Single-Node Setup no Apache CouchDB 3.3 (Criação de _users e _replicator)
curl -X POST http://admin:admin123@localhost:5984/_cluster_setup \
  -H "Content-Type: application/json" \
  -d '{"action": "enable_single_node", "username": "admin", "password": "admin123", "bind_address"
: "0.0.0.0", "port": 5984, "singlenode": true}'

# Inicializar o cluster Couchbase Server 7.6 via CLI com serviços Data, Index e Query
docker exec lab_couchbase couchbase-cli cluster-init \
  -c 127.0.0.1 --cluster-username admin --cluster-password admin123 \
  --services data,index,query --cluster-ramsize 1024 --cluster-index-ramsize 512 --index-storage-s
etting default
```

3. Comandos Práticos no Apache CouchDB (REST, MVCC e Stream)

```
# Criar o banco de dados
curl -X PUT http://admin:admin123@localhost:5984/universidade_db

# Inserir documento com ID fixo -> Devolve "rev": "1-4c61..."
curl -X PUT http://admin:admin123@localhost:5984/universidade_db/aluno_01 \
  -H "Content-Type: application/json" \
  -d '{"nome": "Ana Carolina Silva", "curso": "Ciencia da Computacao", "cra": 8.7}'

# PROVOCAÇÃO DE ERRO MVCC: Atualizar sem _rev -> Dispara HTTP 409 Conflict!
curl -i -X PUT http://admin:admin123@localhost:5984/universidade_db/aluno_01 \
  -H "Content-Type: application/json" -d '{"nome": "Ana Carolina Silva", "cra": 9.1}'

# Atualização correta enviando o _rev -> Sucesso e avanço para rev "2-7051..."
curl -X PUT http://admin:admin123@localhost:5984/universidade_db/aluno_01 \
  -H "Content-Type: application/json" \
  -d '{"_rev": "1-4c6114c65e295552ab1019e2b046b10e", "nome": "Ana Carolina Silva", "cra": 9.1}'

# Escutar feed de alterações contínuo em tempo real (Terminal 2)
curl -N "http://admin:admin123@localhost:5984/universidade_db/_changes?feed=continuous&include;_do
cs=true&since;=now"
```

4. Comandos N1QL (SQL++) no Couchbase Server

```
-- Criação do Bucket via CLI e Scopes/Collections via SQL++
CREATE SCOPE `academico`.`graduacao` IF NOT EXISTS;
CREATE COLLECTION `academico`.`graduacao`.`matriculas` IF NOT EXISTS;

-- Criação de Índices GSI (Primário e Secundário Composto)
CREATE PRIMARY INDEX `idx_primary` ON `academico`.`graduacao`.`matriculas`;
CREATE INDEX `idx_curso_cra` ON `academico`.`graduacao`.`matriculas`(`curso`, `cra` DESC);

-- Consulta Analítica com Operador UNNEST (Achatamento de Array JSON)
SELECT m.nome AS aluno, d.nome AS disciplina, d.nota
FROM `academico`.`graduacao`.`matriculas` AS m
UNNEST m.disciplinas AS d
WHERE m.curso = "Ciencia da Computacao" AND m.cra >= 9.0;
```

MÓDULO 4: AVALIAÇÃO DIDÁTICA UNIVERSITÁRIA

5 Questões inéditas de múltipla escolha com gabarito oficial e justificativas técnicas completas.

Questão 1 — Controle de Concorrência MVCC e Armazenamento no Apache CouchDB

Em uma arquitetura baseada no Apache CouchDB, dois clientes concorrentes (Cliente A e Cliente B) leem simultaneamente o documento '_id': 'pedido_99', cuja revisão é '1-967a00d...'. O Cliente A submete uma requisição PUT com o _rev lido e recebe HTTP 201 Created com nova revisão '2-7051cbe...'. Um milissegundo depois, o Cliente B envia sua requisição PUT para o mesmo documento informando ainda a revisão '1-967a00d...'. O que ocorre internamente?

(A) O CouchDB bloqueia o Cliente B em fila pessimista e sobrescreve no mesmo bloco de disco (in-place update).

(B) O CouchDB rejeita imediatamente o Cliente B com HTTP 409 Conflict, mantendo a escrita do Cliente A gravada de forma append-only no final do arquivo até a execução do Compaction.

(C) O CouchDB aceita ambas as escritas aplicando Last-Write-Wins (LWW) baseado no relógio NTP do servidor.

(D) O CouchDB realiza merge automático dos campos JSON alterados se as propriedades forem distintas.

(E) O CouchDB grava a atualização na memória RAM do vBucket correspondente e agenda a resolução para o Index Service.

Gabarito Oficial: Alternativa (B)

Justificativa Técnica: *O CouchDB implementa MVCC estrito. Ao receber um _rev obsoleto em relação à folha ativa do documento, ele rejeita imediatamente a operação com HTTP 409 Conflict. Por usar armazenamento append-only, os bytes da revisão antiga permanecem em disco até que ocorra um processo de compactação (Compaction).*

Questão 2 — Particionamento por vBuckets e Roteamento no Couchbase Server

Uma equipe de engenharia projeta um sistema financeiro de baixa latência em um cluster Couchbase de 8 nós executando o Data Service. Sobre o mecanismo de particionamento e roteamento KV, assinale a correta:

(A) O Couchbase utiliza proxies roteadores centrais semelhantes ao mongos consultando nós ZooKeeper.

(B) Cada Bucket sofre shard splitting dinâmico a cada 64 MB, exigindo que o cliente consulte a API REST a cada leitura.

(C) Cada Bucket é particionado estaticamente em exatamente 1024 vBuckets, onde o hash CRC32 da chave permite ao SDK consultar seu Cluster Map e conectar-se diretamente ao nó que detém o vBucket ativo via TCP (Single-Hop).

(D) Os vBuckets operam em modo Multi-Primary com todos os 8 nós aceitando escritas simultâneas para a mesma chave.

(E) Quando um novo nó é adicionado, o cluster é obrigado a dobrar o total de vBuckets de 1024 para 2048.

Gabarito Oficial: Alternativa (C)

Justificativa Técnica: *O particionamento em 1024 vBuckets é estático. A função CRC32(chave) % 1024 determina o vBucket. Como o SDK cliente mantém o Cluster Map atualizado em sua própria memória, ele abre conexão TCP direta com o nó onde aquele vBucket está ativo, sem intermediários.*

Questão 3 — Multi-Dimensional Scaling (MDS) e Consultas SQL++ (N1QL)

Qual é a principal vantagem arquitetural da abordagem Multi-Dimensional Scaling (MDS) no Couchbase Server?

(A) Elimina o uso de memória RAM no Data Service, passando todo o tráfego para arquivos de log em disco.

(B) Permite isolar cargas em hardware especializado (Data em RAM, Index em NVMe e Query em CPU) e evita consultas do tipo Scatter-Gather através de índices GSI centralizados.

(C) Obriga que consultas SQL++ sejam pré-compiladas em Design Documents JavaScript como no CouchDB.

(D) Força atualizações bloqueantes e síncronas dos índices secundários no caminho crítico de escrita KV.

(E) Substitui o protocolo binário DCP por chamadas HTTP GET /_changes em texto plano entre os nós.

Gabarito Oficial: Alternativa (B)

Justificativa Técnica: *O MDS desacopla os serviços do banco. Nós de Data focam em memória para operações chave-valor; nós de Query focam em múltiplos núcleos de CPU para resolver joins; e nós de Index mantêm GSIs em SSDs NVMe rápidos, alimentados assincronamente pelo protocolo de streaming DCP sem degradar o tráfego KV.*

Questão 4 — Teorema CAP e Arquitetura Offline-First

Considere: Cenário I (App de inspeção em pistas remotas sem internet com sync local) e Cenário II (Carrinho de compras global com 500k ops/s e consistência imediata). Relacionando ao Teorema CAP e aos dois bancos, assinale a correta:

(A) O Cenário I deve usar Couchbase conectado diretamente ao browser pela porta 11210; o Cenário II deve usar CouchDB.

(B) Ambos os bancos são classificados estritamente como CA, sendo incapazes de operar diante de partições de rede.

(C) O Cenário I alinha-se ao Apache CouchDB (AP), pareado ao PouchDB via HTTP e árvore _rev; o Cenário II alinha-se ao Couchbase (CP por padrão), servindo operações direto da RAM do vBucket ativo.

(D) No Cenário I, o CouchDB descarta edições offline conflitantes sem guardar histórico de ramificação.

(E) No Cenário II, o Couchbase tem menor latência que o Couchbase por gravar diretamente em disco append-only.

Gabarito Oficial: Alternativa (C)

Justificativa Técnica: *O CouchDB é essencialmente AP, perfeito para Offline-First com PouchDB no cliente. O Couchbase adota modelo CP por padrão no nível do vBucket (um único primário ativo por chave), servindo requisições da memória RAM com consistência imediata e latência de microssegundos.*

Questão 5 — Deleção Lógica (Tombstones), Compactação e XDCR

Sobre exclusão de dados e replicação no Apache CouchDB e no Couchbase Server, assinale a alternativa correta:

(A) Deletar um documento no CouchDB apaga fisicamente os bytes de disco na hora, liberando espaço sem Compaction.

(B) No CouchDB, exclusões geram uma nova revisão Tombstone ('_deleted': true) gravada append-only para que a replicação propague a remoção para outros nós em vez de ressuscitar o registro.

(C) O mecanismo XDCR do Couchbase não suporta replicação Active-Active e trava em caso de conflitos concorrentes.

(D) A compactação do CouchDB apaga documentos que não receberam leituras nos últimos 30 dias.

(E) CouchDB e Couchbase exigem downtime total de leituras e escritas para rodar compactação de armazenamento.

Gabarito Oficial: Alternativa (B)

Justificativa Técnica: *Em bancos append-only replicados, deletar dados requer a gravação de um Tombstone ('_deleted': true). Sem esse registro histórico propagado pelo feed de alterações (_changes), nós réplicas desatualizados interpretariam a ausência do dado como uma falha e ressuscitariam o documento.*

MÓDULO 5: PLANO OPERACIONAL DE CONTINGÊNCIA (48 HORAS)

Matriz de papéis e cronograma de ensaio para garantir apresentação impecável de 35 a 45 minutos.

Integrante	Papel de Apresentação	Função Operacional nas 48 Horas
Integrante 1	Slides 1 a 3 (Origem & Filosofia)	Líder de Tempo: Monitora cronômetro e passagens de bastão nos ensaios.
Integrante 2	Slides 4 a 6 (Motor CouchDB)	Revisor Técnico CouchDB: Valida funcionamento de B+Tree, MVCC e _changes.
Integrante 3	Slides 7 a 9 (Motor Couchbase)	Revisor Técnico Couchbase: Valida conceitos de vBuckets, MDS e N1QL.
Integrante 4	Slides 10 e 11 (Mercado CouchDB)	Designer do Deck: Cuida da diagramação e formatação visual dos slides.
Integrante 5	Slides 12 e 13 (Mercado Couchbase)	Co-Designer e Handout: Prepara impressão e distribuição do Quiz de avaliação.
Integrante 6	Slides 14 e 15 (Matriz Comparativa)	Engenheiro de Backup: Grava vídeo de 4 min com comandos do workshop (Plano B).
Integrante 7	Slides 16 a 18 (Workshop Prático)	Piloto de Live Demo: Testa Docker Compose, cURL, portas e script Python.

Checkpoints de Cronômetro da Apresentação (Meta: 41 minutos)

- **Fim do Integrante 1 (Slide 3):** 06:00 acumulados
- **Fim do Integrante 2 (Slide 6):** 12:00 acumulados
- **Fim do Integrante 3 (Slide 9):** 18:00 acumulados
- **Fim do Integrante 4 (Slide 11):** 23:30 acumulados
- **Fim do Integrante 5 (Slide 13):** 29:00 acumulados
- **Fim do Integrante 6 (Slide 15):** 34:30 acumulados
- **Fim do Integrante 7 / Workshop (Slide 18):** 41:00 acumulados (Margem de 4 minutos para perguntas!)